

Addestramento dei Large Language Models

NECKER

15 Luglio 2026

Indice

1	Introduzione: La Logica di Base degli LLM	2
1.1	I Token	2
1.2	Epoche	2
1.3	Gli Embedding	2
1.4	I Parametri (I Pesi della Rete)	3
1.5	La Finestra di Contesto (Context Window)	3
1.6	Fasi di Addestramento: Pre-training e Fine-Tuning	3
2	Il Processo di Addestramento Multi-Stadio	3
2.1	Pre-addestramento (Self-Supervised Pre-training)	3
2.1.1	Configurazione dello Scenario	4
2.1.2	Generazione dei Logits ($\mathbf{z}_2$)	4
2.1.3	Applicazione della Funzione Softmax	4
2.1.4	Il Calcolo dell'Errore: Cross-Entropy Loss	5
2.1.5	Come l'etichetta guida la Backpropagation	5
2.2	Instruction Tuning e Supervised Fine-Tuning (SFT)	6
2.2.1	Configurazione dello Scenario	6
2.3	Reinforcement Learning from Human Feedback (RLHF) ed Allineamento	10
2.3.1	Modellazione della Ricompensa (Reward Modeling)	10
2.3.2	Addestramento del Reward Model (Il Giudice)	10
2.3.3	Correzione Modello: imizzazione PPO e la Regularizzazione KL	14
2.3.4	Evoluzione: Direct Preference Optimization (DPO)	16
2.3.5	Oltre l'RLAIF: Dal Feedback Umano alla Verifica Algoritmica	17

1 Introduzione: La Logica di Base degli LLM

I Large Language Models (LLM) **prevedono il pezzo di testo successivo** basandosi sul contesto che hanno appena elaborato. Si tratta, in sostanza, di una versione estremamente potente e complessa della funzione di autocompletamento dei nostri smartphone.

Questo meccanismo è noto come *generazione autoregressiva*: il modello analizza un testo, calcola le probabilità per la parola che dovrebbe seguire, ne seleziona una, la "incolla" al testo originale e ricomincia il processo da capo. È un ciclo continuo di lettura e predizione.

1.1 I Token

Per far funzionare questo meccanismo in modo efficiente, i modelli non leggono le frasi lettera per lettera, e nemmeno parola per parola. Usano un'unità di misura chiamata **token**.

Immagina i token come dei mattoncini Lego di dimensioni variabili che compongono il linguaggio. La necessità di usarli nasce da due problemi pratici fondamentali:

- **Perché non lettera per lettera?** Se il modello leggesse carattere per carattere ("a-l-b-e-r-o"), ogni singola unità non avrebbe alcun significato semantico da sola. Inoltre, per leggere un breve paragrafo dovrebbe elaborare migliaia di lettere. Poiché l'architettura su cui si basano (il Transformer) fa molta fatica (richiede una potenza di calcolo quadratica) quando il testo si allunga, usare le singole lettere renderebbe il modello lentissimo e incapace di ricordare il contesto a lungo termine.
- **Perché non parola per parola?** Se usassimo parole intere, il vocabolario del modello dovrebbe contenere ogni singola parola esistente: tutti i verbi coniugati, i plurali, le parole straniere, i neologismi e gli errori di battitura. Avremmo un dizionario di milioni di voci. Una tabella del genere richiederebbe una quantità di memoria (VRAM) semplicemente insostenibile per qualsiasi hardware attuale.

Il compromesso perfetto: I token (o *sub-words*) risolvono entrambi i problemi. Le parole molto comuni (come "casa" o "gatto") corrispondono a un singolo token. Le parole rare o lunghe vengono invece "spezzettate" in più token. Ad esempio, la parola "psiconeuroendocrinologia" potrebbe essere divisa nei token "psico", "neuro", "endocrino", "logia". In questo modo, con un dizionario fisso e compatto (di solito tra i 30.000 e i 100.000 token), il modello può comporre e comprendere qualsiasi testo, assemblando frammenti noti per gestire parole che non ha mai visto.

1.2 Epoche

Un'**epoca** è definita come un passaggio completo dell'intero dataset di addestramento attraverso il modello. Significa mostrare al modello ogni singolo dato esattamente una volta per fargli aggiornare i pesi. Passare più volte lo stesso dataset (più epoche) serve a far "fissare" le nozioni, ma c'è un limite: esagerare porta all'overfitting (il modello memorizza i dati a memoria invece di capire la logica).

1.3 Gli Embedding

I token, di per sé, sono solo numeri interi (ID) che puntano a una riga di un vocabolario, ma un numero non ha alcun significato semantico per una rete neurale. Per risolvere questo problema, ogni token viene convertito in un **Embedding**.

Un embedding è un vettore denso di numeri reali (spesso composto da migliaia di dimensioni, es. $d = 4096$). L'aspetto rivoluzionario degli embedding è che traducono la semantica in geometria: parole con significati simili (come "re" e "regina") avranno vettori posizionati molto vicini in questo spazio multidimensionale ad alta dimensionalità. In questo modo, il modello non elabora parole, ma compie vere e proprie operazioni algebriche sui concetti.

1.4 I Parametri (I Pesi della Rete)

Quando si parla di modelli da "7 miliardi" o "70 miliardi" di parametri (come Llama 3 o GPT-4), ci si riferisce ai **pesi** e ai *bias* delle matrici all'interno della rete neurale. I parametri sono i "nodi di memoria" del modello. Inizialmente sono impostati con valori casuali. Durante l'addestramento, ogni volta che il modello cerca di indovinare la parola successiva e sbaglia, un algoritmo matematico (la *Backpropagation*) aggiusta leggermente questi miliardi di numeri per ridurre l'errore al tentativo successivo. L'intelligenza di un LLM risiede interamente nella configurazione finale di queste immense matrici di numeri reali.

1.5 La Finestra di Contesto (Context Window)

Il meccanismo autoregressivo non ha una memoria infinita. Il modello può "guardare indietro" solo per un numero massimo di token alla volta: questo limite è la **Finestra di Contesto** (*Context Window*). Se un modello ha una finestra di 8.000 token (circa 6.000 parole), una volta superato quel limite inizierà a "dimenticare" l'inizio della conversazione o del documento. Dal punto di vista computazionale, raddoppiare la finestra di contesto richiede molta più RAM (VRAM) e potenza di calcolo (spesso con un costo quadratico), motivo per cui l'ottimizzazione di questo parametro è una delle sfide ingegneristiche più complesse.

1.6 Fasi di Addestramento: Pre-training e Fine-Tuning

L'addestramento di un LLM moderno non avviene in un unico blocco, ma si divide in due fasi cruciali:

- **Pre-training (Addestramento Base):** È la fase più massiccia e costosa. Il modello "legge" terabyte di dati grezzi da internet (libri, articoli, codice sorgente) per mesi, imparando la grammatica, la logica, la programmazione e i fatti del mondo. In questa fase, impara semplicemente a prevedere la parola successiva in modo non strutturato.
- **Fine-Tuning (Allineamento):** Un modello pre-addestrato non sa rispondere alle domande, sa solo completare frasi. Il Fine-Tuning (spesso eseguito tramite tecniche come il *Supervised Fine-Tuning* o l'apprendimento per rinforzo) utilizza dataset molto più piccoli e curati per insegnare al modello a comportarsi come un assistente utile, a seguire le istruzioni (prompt) e a rispondere in formato chat.

2 Il Processo di Addestramento Multi-Stadio

L'addestramento di un LLM allo stato dell'arte si articola in una pipeline rigidamente sequenziale volta a trasformare un approssimatore statistico di stringhe in un assistente vero e proprio.

2.1 Pre-addestramento (Self-Supervised Pre-training)

Il pre-addestramento rappresenta la fase computazionalmente più intensiva (spesso assorbendo oltre il 98% del budget di calcolo totale, misurato in $10^{21} \sim 10^{24}$ FLOPs (**numero di Floating Point Operations**)). L'obiettivo è costringere la rete ad apprendere la sintassi, la semantica, il ragionamento logico di base e la conoscenza enciclopedica mediante la semplice predizione del token successivo.

1. Cura del Dato e Flusso Computazionale:

Il modello viene esposto a dataset di dimensioni colossali (nell'ordine dei trilioni di token), composti da frammenti eterogenei del web (es. Common Crawl), libri, articoli scientifici e codice sorgente. Prima del training, questi dati subiscono un filtraggio euristico rigoroso (de-duplicazione, rimozione di artefatti e contenuti a bassa qualità) per evitare che il modello memorizzi rumore. Durante la *forward pass*, data una sequenza di input di lunghezza T , $X = (x_1, \dots, x_T)$, il modello produce per ogni posizione i un vettore di *logits* $\mathbf{z}_i \in \mathbb{R}^{|\mathcal{V}|}$, dove $|\mathcal{V}|$ è la dimensione del vocabolario. La probabilità predetta per il token successivo si ottiene applicando la funzione softmax:

$$P_{\theta}(x_i | x_1, \dots, x_{i-1}) = \frac{\exp(z_{x_i})}{\sum_{k=1}^{|\mathcal{V}|} \exp(z_k)} \quad (1)$$

Esempio Pratico: Calcolo dei Logits e Softmax

Per comprendere il comportamento del modello durante la *forward pass*, analizziamo uno scenario semplificato.

2.1.1 Configurazione dello Scenario

- **Vocabolario ($\mathcal{V}$):** Il modello conosce solo 4 parole: $\mathcal{V} = \{\text{"gatto"}, \text{"casa"}, \text{"mangia"}, \text{"topo"}\}$. La dimensione del vocabolario è $|\mathcal{V}| = 4$.
- **Sequenza di input (X):** Data la frase *"Il gatto"*, composta da $T = 2$ token, vogliamo predire il token successivo alla posizione $i = 2$ (ovvero dopo la parola "gatto").
- **Etichetta:** Per ogni parola da prevedere viene fornito un vettore **One-Hot** della stessa dimensione del vocabolario detto etichetta (un vettore composto da tutti 0 tranne un uno sulla parola corretta).

$$\begin{aligned}y_{\text{gatto}} &= 0 \\y_{\text{casa}} &= 0 \\y_{\text{mangia}} &= 1 \\y_{\text{topo}} &= 0\end{aligned}$$

Il vettore dell'etichetta è quindi:

$$\mathbf{y} = [0, \quad 0, \quad 1, \quad 0]$$

2.1.2 Generazione dei Logits ($\mathbf{z}_2$)

All'ultimo layer lineare, il modello calcola un vettore di punteggi grezzi non normalizzati, denominati *logits*. Per la posizione $i = 2$, assumiamo che il vettore $\mathbf{z}_2 \in \mathbb{R}^4$ sia:

$$\mathbf{z}_2 = [-1.2, \quad 0.5, \quad 4.2, \quad 1.8]$$

Ogni componente corrisponde a una parola del vocabolario:

$$\begin{aligned}z_{2,\text{gatto}} &= -1.2 \\z_{2,\text{casa}} &= 0.5 \\z_{2,\text{mangia}} &= 4.2 \\z_{2,\text{topo}} &= 1.8\end{aligned}$$

2.1.3 Applicazione della Funzione Softmax

La probabilità per ciascun token k nel vocabolario si ottiene applicando la funzione softmax:

$$P(\text{token}_k \mid \text{input}) = \frac{e^{z_{2,k}}}{\sum_{j=1}^{|\mathcal{V}|} e^{z_{2,j}}}$$

Passo A: Calcolo degli esponenziali
($e^{z_{2,k}}$)

- $e^{-1.2} \approx 0.301$
- $e^{0.5} \approx 1.649$
- $e^{4.2} \approx 66.686$
- $e^{1.8} \approx 6.050$

Passo B: Calcolo del denominatore (Somma)

$$\sum_{j=1}^4 e^{z_{2,j}} = 0.301 + 1.649 + 66.686 + 6.050 = 74.686$$

Passo C: Calcolo delle probabilità finali

Dividendo ciascun esponenziale per la somma totale, otteniamo la distribuzione di probabilità normalizzata:

- $P(\text{"gatto"}) = \frac{0.301}{74.686} \approx 0.004 \implies \mathbf{0.4\%}$
- $P(\text{"casa"}) = \frac{1.649}{74.686} \approx 0.022 \implies \mathbf{2.2\%}$
- $P(\text{"mangia"}) = \frac{66.686}{74.686} \approx 0.893 \implies \mathbf{89.3\%}$
- $P(\text{"topo"}) = \frac{6.050}{74.686} \approx 0.081 \implies \mathbf{8.1\%}$

Il vettore di output della softmax è:

$$\mathbf{p} = [0.004, \quad 0.022, \quad 0.893, \quad 0.081]$$

La somma delle probabilità è esattamente pari a 1 ($\sum p_k = 1$). In questa fase, il modello assegna la probabilità più alta (89.3%) al token "mangia".

2.1.4 Il Calcolo dell'Errore: Cross-Entropy Loss

Ora che abbiamo sia il vettore delle probabilità predette sia il vettore dell'etichetta reale ($\mathbf{y}$), possiamo calcolare la *Loss* (l'errore) usando la funzione di **Cross-Entropy**:

$$\mathcal{L} = - \sum_{k=1}^{|\mathcal{V}|} y_k \log(p_k)$$

Sostituendo i nostri valori nella formula:

$$\mathcal{L} = - \left(0 \cdot \log(0.004) + 0 \cdot \log(0.022) + 1 \cdot \log(0.893) + 0 \cdot \log(0.081) \right)$$

Poiché tutti gli zeri annullano le parole sbagliate, il calcolo si riduce semplicemente a:

$$\mathcal{L} = -\log(0.893) \approx 0.113$$

2.1.5 Come l'etichetta guida la Backpropagation

Il valore di Loss ottenuto (0.113) è il punto di partenza per la *backpropagation*.

- Se il modello avesse predetto "mangia" con il 100% di probabilità ($p_k = 1$), la Loss sarebbe stata $-\log(1) = 0$ (errore nullo, nessun aggiornamento dei pesi).
- Se il modello avesse sbagliato completamente, predicendo ad esempio "casa" e dando a "mangia" una probabilità vicina allo zero (es. 0.001), la Loss sarebbe schizzata a $-\log(0.001) = 6.9$ (errore altissimo).

Questo numero d'errore viene derivato matematicamente rispetto a tutti i pesi del modello. Il gradiente risultante spingerà i pesi delle matrici Query, Key, Value e i vettori di embedding in una direzione ben precisa: fare in modo che, la prossima volta che il modello vedrà l'input "Il gatto", il logit di "mangia" sia ancora più alto e la sua probabilità si avvicini sempre di più a quell'1 indicato dall'etichetta.

2.2 Instruction Tuning e Supervised Fine-Tuning (SFT)

Finita la fase di pre addestramento che come detto precedentemente occupa circa il 98% del budget di calcolo totale, otteniamo un *base model* il quale è un eccellente completatore di documenti, ma non è un assistente. Se ricevesse in input "Come si fa la pizza?", potrebbe statisticamente rispondere con un'altra domanda come "Come si fa il pane?". Per mutare questo comportamento da una distribuzione incondizionata a una distribuzione condizionata dall'istruzione, si introduce il **Supervised Fine-Tuning (SFT)**.

Formattazione e Mascheramento (Loss Masking):

Il modello viene addestrato su un dataset altamente curato (decine o centinaia di migliaia di esempi, non miliardi) contenente prompt umani e risposte di alta qualità scritte da esperti. I dati vengono formattati in strutture rigide tramite token speciali (es. `<|user|>`, `<|assistant|>`). La differenza cruciale rispetto al pre-addestramento risiede nel calcolo del gradiente: non vogliamo che il modello "impari a generare" le istruzioni dell'utente, ma solo le risposte. Pertanto, la loss sui token appartenenti all'istruzione (il *prompt* I) viene azzerata (*masked*). Dato un esempio composto dal prompt $I = (p_1, \dots, p_N)$ e dalla risposta $R = (r_1, \dots, r_M)$, la loss agisce solo sui token r_j :

$$\mathcal{L}_{SFT}(\theta) = -\frac{1}{M} \sum_{j=1}^M \log P_{\theta}(r_j \mid I, r_1, \dots, r_{j-1}) \quad (2)$$

Esempio Pratico di SFT e Mascheramento della Loss (Loss Masking)

2.2.1 Configurazione dello Scenario

- **Prompt dell'utente (I):** `<|user|> "Dimmi casa" <|assistant|>`
- **Risposta target (R):** `"Casa"`

Ipotizziamo che la sequenza intera venga spezzata nei seguenti 5 token:

$$X = (\underbrace{x_1}_{\text{<|user|>}}, \underbrace{x_2}_{\text{"Dimmi"}}, \underbrace{x_3}_{\text{"casa"}}, \underbrace{x_4}_{\text{<|assistant|>}}, \underbrace{x_5}_{\text{"Casa"}})$$

In questo scenario, la lunghezza del prompt è $N = 4$ e la lunghezza della risposta attesa è $M = 1$.

Passo A: Il Forward Pass sulla sequenza completa

Il modello elabora l'intera sequenza e genera una distribuzione di probabilità per il token successivo in ogni singola posizione. Supponiamo che le probabilità assegnate dal modello ai token che *effettivamente* compaiono subito dopo siano:

1. Alla posizione 1 (dopo `<|user|>`), la probabilità di generare "Dimmi" è $P(x_2 \mid x_1) = 0.45$
2. Alla posizione 2 (dopo "Dimmi"), la probabilità di generare "casa" è $P(x_3 \mid x_1, x_2) = 0.70$
3. Alla posizione 3 (dopo "casa"), la probabilità di generare `<|assistant|>` è $P(x_4 \mid x_1, x_2, x_3) = 0.90$
4. Alla posizione 4 (dopo `<|assistant|>`), la probabilità di generare il primo token della risposta ("Casa") è $P(r_1 \mid I) = 0.60$

Passo B: Applicazione del Loss Masking

Se applicassimo il pre-addestramento standard, calcoleremmo la Cross-Entropy su tutte e 4 le posizioni. Nel SFT, invece, creiamo un vettore di mascheramento che assegna peso 0 ai token del prompt e peso 1 ai token della risposta dell'esperto:

$$\text{Mask} = [0, \quad 0, \quad 0, \quad 1]$$

Andiamo a calcolare la Loss per ogni posizione ($\mathcal{L}_i = -\log P$):

- $\mathcal{L}_1 = -\log(0.45) \approx 0.798 \implies \text{Mascherata} \rightarrow \mathbf{0}$
- $\mathcal{L}_2 = -\log(0.70) \approx 0.356 \implies \text{Mascherata} \rightarrow \mathbf{0}$
- $\mathcal{L}_3 = -\log(0.90) \approx 0.105 \implies \text{Mascherata} \rightarrow \mathbf{0}$
- $\mathcal{L}_4 = -\log(0.60) \approx 0.510 \implies \text{Attiva} \rightarrow \mathbf{0.510}$

Passo C: Calcolo della $\mathcal{L}_{SFT}(\theta)$

Applicando la formula, sommiamo solo gli errori dei token della risposta ($M = 1$) e ne facciamo la media:

$$\mathcal{L}_{SFT}(\theta) = -\frac{1}{1} \sum_{j=1}^1 \log P_{\theta}(r_j \mid I, r_1, \dots, r_{j-1}) = 0.510$$

Come si nota dal calcolo, gli errori commessi dal modello nel prevedere le parole dell'utente ($\mathcal{L}_1, \mathcal{L}_2, \mathcal{L}_3$) vengono completamente azzerati e ignorati.

In questo modo, durante la backpropagation, i gradienti non modificheranno i pesi del modello per insegnargli a scrivere come un utente umano o a imitare i suoi prompt. L'intero segnale di aggiornamento deriverà esclusivamente da $\mathcal{L}_4 = 0.510$. Il modello viene così forzato matematicamente a focalizzare tutta la sua capacità di apprendimento sull'imparare a generare risposte utili, corrette e formattate nello stile dell'assistente perfetto.

1. Quando viene calcolata la Loss:

Matematicamente, la loss viene calcolata **su ogni singolo token della risposta (parola per parola)**, e non solo alla fine. Tuttavia, l'aggiornamento dei pesi (backpropagation) non avviene dopo ogni singola parola, ma viene accumulato alla fine della sequenza (o del batch).

Prendiamo il tuo esempio:

$$\text{Sequenza} = (\underbrace{\langle \text{user} \rangle, \dots, \langle \text{assistant} \rangle}_{\text{Prompt Mascherato (Loss} = 0)}, \underbrace{r_1}_{\text{"casa"}}, \underbrace{r_2}_{\text{"e"}}, \underbrace{r_3}_{\text{"patate"}})$$

Durante il forward pass, il modello calcola **contemporaneamente** in parallelo i logits e la softmax per ogni singola posizione della risposta. Otterremo quindi tre distribuzioni di probabilità distinte:

- $P(r_1 \mid I) \implies$ probabilità che dopo il prompt ci sia "casa".
- $P(r_2 \mid I, r_1) \implies$ probabilità che dopo "casa" ci sia "e".
- $P(r_3 \mid I, r_1, r_2) \implies$ probabilità che dopo "casa e" ci sia "patate".

Ognuna di queste tre predizioni genera una sua Cross-Entropy Loss parziale confrontandosi con la rispettiva etichetta del dataset:

$$\mathcal{L}_1 = -\log P(\text{"casa"}), \quad \mathcal{L}_2 = -\log P(\text{"e"}), \quad \mathcal{L}_3 = -\log P(\text{"patate"})$$

La Loss totale dell'SFT per questo esempio sarà la **media aritmetica** di queste tre loss individuali:

$$\mathcal{L}_{SFT} = \frac{\mathcal{L}_1 + \mathcal{L}_2 + \mathcal{L}_3}{3}$$

Se il modello indovina "casa" ed "e" ma sbaglia catastroficamente su "patate" (ad esempio predicendo "gatti"), la loss $\mathcal{L}_3$ sarà altissima, facendo alzare la media globale. Quando l'errore medio $\mathcal{L}_{SFT}$ viene retropropagato, il gradiente conterrà le informazioni per correggere l'intera catena di calcolo,

focalizzandosi maggiormente sui parametri che hanno causato l'errore specifico sulla parola "patate".

ATTENZIONE: come detto all'inizio, i pesi non vengono corretti dopo ogni singola parola. Se modificassimo i pesi del Transformer dopo aver calcolato l'errore su "casa", le matrici cambierebbero durante il calcolo della sequenza stessa. Questo distruggerebbe il parallelismo di cui i Transformer hanno bisogno per correre veloci sulle GPU.

Il processo reale funziona così:

1. **Forward Pass:** Il modello calcola in parallelo tutte le probabilità e tutte le loss parziali ($\mathcal{L}_1, \mathcal{L}_2, \mathcal{L}_3$).
2. **Accumulo/Media:** Si calcola la Loss totale media $\mathcal{L}_{SFT} = \frac{\mathcal{L}_1 + \mathcal{L}_2 + \mathcal{L}_3}{3}$.
3. **Backward Pass (Backpropagation):** Si calcola il gradiente complessivo e solo alla fine si aggiornano i pesi del modello in un colpo solo.

Perché l'errore non si "spalma" in modo uguale?

Se la Loss è una media, verrebbe da pensare che l'aggiornamento colpisca tutto indistintamente. Invece non è così, grazie alla **Regola della Catena (Chain Rule)** delle derivate.

La Loss totale rispetto a un generico peso w del modello è la derivata della media:

$$\frac{\partial \mathcal{L}_{SFT}}{\partial w} = \frac{1}{3} \left(\frac{\partial \mathcal{L}_1}{\partial w} + \frac{\partial \mathcal{L}_2}{\partial w} + \frac{\partial \mathcal{L}_3}{\partial w} \right)$$

Ciascun termine $\frac{\partial \mathcal{L}_i}{\partial w}$ indica quanto quel peso w ha contribuito all'errore della parola i -esima. Vediamo cosa succede numericamente nei due scenari estremi del tuo esempio:

- **Caso 1: Predizione ottima (es. su $\mathcal{L}_1$ per "casa" e $\mathcal{L}_2$ per "e")**

Se il modello era già sicurissimo di queste parole, la loro probabilità è vicina a 1 (es. 0.99). La loro loss è quasi zero, e matematicamente la derivata di una loss vicina a zero è quasi zero:

$$\frac{\partial \mathcal{L}_1}{\partial w} \approx 0, \quad \frac{\partial \mathcal{L}_2}{\partial w} \approx 0$$

Questo significa che quelle parole non inviano quasi nessun segnale di correzione a ritroso.

- **Caso 2: Errore catastrofico (es. su $\mathcal{L}_3$ per "patate")**

Se il modello assegna a "patate" una probabilità bassissima (es. 0.01), la loss $\mathcal{L}_3$ esplode. La derivata di una loss alta rispetto ai logits è **grandissima** (matematicamente, la derivata della cross-entropy rispetto al logit della parola corretta è proporzionale a $P_{predetta} - 1$, che nel nostro caso fa $0.01 - 1 = -0.99$, un valore d'impatto massimo).

$$\frac{\partial \mathcal{L}_3}{\partial w} \gg 0$$

Sostituendo questi comportamenti nella formula del gradiente totale:

$$\frac{\partial \mathcal{L}_{SFT}}{\partial w} = \frac{1}{3} \left(0 + 0 + \frac{\partial \mathcal{L}_3}{\partial w} \right) = \frac{1}{3} \frac{\partial \mathcal{L}_3}{\partial w}$$

In Conclusione:

Come vedi, anche se c'è un fattore di riduzione (la divisione per 3), l'intero valore del gradiente finale è dominato **esclusivamente** dall'errore commesso su "patate".

Quando questo gradiente viaggia all'indietro nella rete, andrà a modificare pesantemente proprio quelle attivazioni, quei vettori di attenzione e quei pesi delle matrici Query, Key e Value che si sono attivati alla posizione 3 e che hanno causato la deviazione della risposta. I pesi che hanno calcolato correttamente "casa" ed "e" riceveranno un gradiente nullo o infinitesimale, rimanendo di fatto stabili.

2. Come il modello ricorda le parole passate:

Quando il modello deve predire la seconda parola, non genera un "vettore speciale" da passarsi a mano, bensì la comprensione avviene grazie al **meccanismo di attenzione** e al contesto che si allunga. I modelli di linguaggio sono **autoregressivi**, infatti l'output generato al passo precedente diventa parte dell'input del passo successivo.

3. Scopo di SFT:

Durante l'SFT, l'obiettivo non è iniettare nuova conoscenza nel modello (operazione inefficiente in questa fase e incline a generare *allucinazioni*), ma piuttosto insegnargli il "formato" di output desiderato, l'estrazione logica e il tono. Per evitare il *catastrophic forgetting* (la perdita delle capacità generali acquisite nel pre-training), l'SFT impiega:

- Un learning rate molto più basso (tipicamente un decimo o un centesimo di quello massimo usato nel pre-training).
- Un numero estremamente limitato di epoche (spesso da 1 a 3), per evitare che il modello vada in overfitting sullo stile delle risposte umane perdendo entropia generativa (nella fase di **pre-training** generalmente si fa una sola epoca).

In contesti in cui le risorse computazionali sono limitate, questa fase può essere implementata tramite tecniche di **Parameter-Efficient Fine-Tuning (PEFT)**, come **LoRA** (Low-Rank Adaptation), che congela la matrice dei pesi originali θ_{pre} e aggiorna solo matrici di rango inferiore additive, riducendo drasticamente la memoria necessaria per calcolare i gradienti.

2.3 Reinforcement Learning from Human Feedback (RLHF) ed Allineamento

Sebbene l'SFT migliori l'aderenza alle istruzioni, non garantisce l'allineamento assiologico del modello rispetto a criteri di utilità, accuratezza e sicurezza (*Helpfulness, Honesty, Harmlessness*). Per ovviare a ciò, si introduce il Reinforcement Learning basato sull'interazione umana.

2.3.1 Modellazione della Ricompensa (Reward Modeling)

Inizialmente, viene addestrato un modello di ricompensa $R_\phi(x, y)$, parametrizzato da pesi ϕ , che accetta come input un prompt x e una risposta y e restituisce uno scalare in $\mathbb{R}$ indicante la qualità della risposta. L'addestramento sfrutta un dataset di preferenze umane in cui, per ogni prompt x , un valutatore ha preferito la risposta y_w (*winning*) rispetto a y_l (*losing*). Assumendo il modello di scelta di Bradley-Terry, la loss del Reward Model è definita come:

$$\mathcal{L}_{\text{Reward}}(\phi) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(R_\phi(x, y_w) - R_\phi(x, y_l))] \quad (3)$$

dove $\sigma(z) = \frac{1}{1+e^{-z}}$ è la funzione sigmoide.

2.3.2 Addestramento del Reward Model (Il Giudice)

A differenza della policy (l'LLM generativo), il Reward Model non deve produrre una distribuzione di probabilità su un intero vocabolario per scegliere la parola successiva, ma deve restituire un singolo valore numerico continuo (uno scalare in $\mathbb{R}$). Per questo motivo per ottenere un modello in grado di valutare, viene modificato **llm** di partenza (generalmente una copia del modello dopo la fase **SFT**).

A. Modifiche Architettureali: Dalla Generazione alla Regressione

Per trasformare un modello di linguaggio standard in un Reward Model, si applica un'operazione chirurgica all'architettura di rete:

1. Si prende un LLM pre-addestrato.
2. Si rimuove l'ultimo livello lineare (la testa di classificazione), che proietta i vettori interni sulla dimensione del vocabolario $|\mathcal{V}|$ per il calcolo della softmax.
3. Al suo posto, si innesta una **testa di regressione** (*Value Head*), ovvero un singolo livello lineare con un unico neurone di output senza alcuna funzione di attivazione finale.
4. Il modello accetta come input la sequenza concatenata del prompt e della risposta (x, y) e restituisce un unico valore numerico continuo:

$$R_\phi(x, y) \in \mathbb{R}$$

ATTENZIONE: poiché il modello elabora token per token e qui necessitiamo una sola risposta finale, nella realtà si fa scorrere il modello fino all'elaborazione dell'ultimo token e si fa la previsione su di esso dato che contiene l'informazione di tutti i precedenti. (questo passaggio è chiamato *last-token pooling*)

Esempio Numerico di Last-Token Pooling

1. La Sequenza di Input

Consideriamo il prompt dell'utente x e la risposta del modello y concatenati in un'unica sequenza di $T = 4$ token complessivi (inclusi i token speciali):

$$X = (\underbrace{x_1}_{<|user|>}, \underbrace{x_2}_{\text{"Grazie"}}, \underbrace{x_3}_{<|assistant|>}, \underbrace{x_4}_{\text{"Prego"}})$$

Ipotizziamo che la dimensione dei vettori nascosti estratti dal Transformer sia $d = 3$ (nella realtà sono migliaia, es. 4096).

2. Generazione degli Hidden States

Il Transformer elabora la sequenza. All'ultimo strato, prima della testa di regressione, produce un vettore nascosto $\mathbf{h}_t \in \mathbb{R}^3$ per ciascuno dei 4 token. Poiché l'attenzione è causale, ogni vettore accumula la storia della frase fino a quel momento:

$$\begin{aligned}\mathbf{h}_1 &= [0.1, \quad -0.2, \quad 0.4]^T \quad (\text{rappresenta solo } \langle \text{user} \rangle) \\ \mathbf{h}_2 &= [0.5, \quad 0.1, \quad -0.1]^T \quad (\text{rappresenta } \langle \text{user} \rangle \text{ Grazie}) \\ \mathbf{h}_3 &= [0.2, \quad 0.8, \quad 0.3]^T \quad (\text{rappresenta } \langle \text{user} \rangle \text{ Grazie } \langle \text{assistant} \rangle) \\ \mathbf{h}_4 &= [0.9, \quad -1.2, \quad 1.5]^T \quad (\text{rappresenta l'intera conversazione completata})\end{aligned}$$

3. Applicazione della Value Head ($\mathbf{w}$)

La testa di regressione è rappresentata da un singolo vettore di pesi $\mathbf{w} \in \mathbb{R}^3$ che mappa lo spazio tridimensionale in un unico numero reale. Supponiamo che i pesi della nostra testa (addestrati) siano:

$$\mathbf{w} = [2.0, \quad 1.0, \quad -0.5]^T$$

Moltiplichiamo linearmente (prodotto scalare $\mathbf{w}^T \mathbf{h}_t$) ciascun vettore nascosto per ottenere il punteggio parziale r_t in ogni posizione t :

- **Al token 1:**

$$r_1 = (2.0 \cdot 0.1) + (1.0 \cdot -0.2) + (-0.5 \cdot 0.4) = 0.2 - 0.2 - 0.2 = -0.2$$

- **Al token 2:**

$$r_2 = (2.0 \cdot 0.5) + (1.0 \cdot 0.1) + (-0.5 \cdot -0.1) = 1.0 + 0.1 + 0.05 = 1.15$$

- **Al token 3:**

$$r_3 = (2.0 \cdot 0.2) + (1.0 \cdot 0.8) + (-0.5 \cdot 0.3) = 0.4 + 0.8 - 0.15 = 1.05$$

- **Al token 4 (Ultimo Token):**

$$r_4 = (2.0 \cdot 0.9) + (1.0 \cdot -1.2) + (-0.5 \cdot 1.5) = 1.8 - 1.2 - 0.75 = -0.15$$

Il modello ha generato internamente un vettore di punteggi intermedi:

$$\mathbf{r} = [-0.2, \quad 1.15, \quad 1.05, \quad -0.15]$$

4. Estrazione del Punteggio Finale (Pooling)

Il sistema di addestramento applica la maschera di selezione per isolare l'ultimo elemento ($t = T = 4$), che corrisponde al momento in cui il modello ha "finito di leggere" tutto l'input:

$$R_\phi(x, y) = r_4 = -0.15$$

I valori r_1, r_2, r_3 vengono completamente scartati. Il valore -0.15 viene preso come la valutazione numerica finale (lo scalare) della risposta dell'assistente. Questo numero sarà la base per calcolare la Loss di Bradley-Terry o per guidare l'aggiornamento dell'attore nel PPO.

B. Costruzione del Dataset di Preferenze

Poiché chiedere a valutatori umani di assegnare punteggi numerici e coerenti a testi complessi è un compito cognitivamente instabile (per me una risposta potrebbe valere 8, per altri 5), l'addestramento si basa su **confronti a coppie (pairwise comparisons)**. Dato un prompt x , il modello SFT genera due diverse risposte alternative. Un annotatore umano esprime una preferenza binaria, indicando quale sia la risposta migliore (y_w , *winning*) e quale la peggiore (y_l , *losing*).

Il dataset risultante $\mathcal{D}$ è composto da triplette:

$$\mathcal{D} = \{(x, y_w, y_l)_1, \dots, (x, y_w, y_l)_K\}$$

C. Formulazione della Loss basata sul Modello di Bradley-Terry

Definita la struttura del modello e il dataset, per addestrare i parametri ϕ del Reward Model sfruttiamo il modello probabilistico di **Bradley-Terry**. Questo modello assume che la probabilità che un valutatore preferisca la risposta y_w rispetto a y_l dipenda unicamente dalla differenza tra i rispettivi punteggi latenti calcolati dal network (se una risposta cattiva prende 98 e quella buona 100, non ha differenza rispetto a cattiva = 0 e buona = 2):

$$P(y_w \succ y_l | x) = \sigma(R_\phi(x, y_w) - R_\phi(x, y_l))$$

dove $\sigma(z) = \frac{1}{1+e^{-z}}$ è la funzione sigmoide, che mappa qualsiasi differenza reale nell'intervallo (0,1) interpretabile come probabilità.

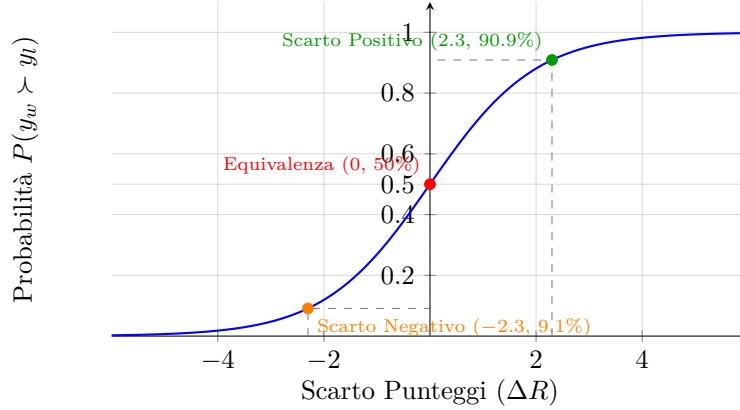


Figura 1: Grafico della funzione sigmoide applicata al modello di Bradley-Terry. I tre punti evidenziano come la differenza tra i punteggi del Reward Model (ΔR) si traduca direttamente nella probabilità di preferenza della risposta ottimale.

ATTENZIONE: se il modello fa bene percentuale alta (piccolo errore), se il modello fa male percentuale bassa (errore grande), se il modello reputa le risposte uguali percentuale media (errore medio). In quest'ultimo caso il modello verrebbe corretto di molto anche se le risposte nel dataset fossero davvero molto simili, questo non è un problema perché statisticamente, il 50% dei valutatori umani sceglieranno una mentre il resto l'altra, compensando quindi prima da un lato poi dall'altro.

Per questo motivo si è introdotta una variante per non sprecare capacità computazionale, introducendo una soglia m sulla differenza dei punteggi, in modo che al di sotto di essa non si applichi la correzione. L'obiettivo dell'addestramento è quindi rendere il modello abile di dare il corretto punteggio alle risposte:

$$\mathcal{L}_{Reward}(\phi) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(R_\phi(x, y_w) - R_\phi(x, y_l))]$$

- **– (Segno Meno Iniziale):** Poiché vogliamo *massimizzare* la probabilità che il modello assegni il punteggio corretto, dovremmo teoricamente usare un algoritmo di salita del gradiente. Tuttavia, per convenzione, gli ottimizzatori di deep learning sono progettati per *minimizzare* funzioni di perdita. Il segno meno inverte il problema: massimizzare la probabilità (verosimiglianza) equivale matematicamente a minimizzare la sua log-verosimiglianza negativa.
- $\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}}$ **(Valore Atteso o Media):** Rappresenta l'operatore di valore atteso (la media matematica). Indica che la loss non viene calcolata su un singolo esempio isolato, ma viene mediata su tutte le triplette (x, y_w, y_l) campionate dal dataset delle preferenze umane $\mathcal{D}$. Nella pratica del codice, questa media viene calcolata ad ogni passo di addestramento su un sottoinsieme di dati chiamato *mini-batch*.
- $(x, y_w, y_l) \sim \mathcal{D}$ **(Campionamento dal Dataset):** Specifica la struttura dei dati di addestramento estratti dal dataset delle preferenze $\mathcal{D}$.

- $R_\phi(x, y_w) - R_\phi(x, y_l)$ (**Differenza dei Punteggi o ΔR**): Rappresenta lo scarto numerico (il divario) calcolato dal Reward Model tra la risposta buona e la risposta cattiva.
 - Se lo scarto è ampiamente positivo, significa che il Giudice sta assegnando un punteggio corretto, premiando y_w .
 - Se lo scarto è vicino a zero, significa che il Giudice non vede differenze qualitative tra le due risposte.
 - Se lo scarto è negativo, significa che il Giudice sta commettendo un errore grossolano, valutando la risposta dannosa y_l meglio di quella corretta y_w .
- $\sigma(\cdot)$ (**Funzione Sigmoidale**): È la funzione di attivazione definita come $\sigma(z) = \frac{1}{1+e^{-z}}$.
- $\log(\cdot)$ (**Logaritmo Naturale**): Viene applicato alla probabilità restituita dalla sigmoide. Il logaritmo serve a stabilizzare numericamente il calcolo e a "punire" gli errori in modo non lineare:
 - Se la probabilità è vicina a 1 (modello sicuro e corretto), $\log(1) = 0$, determinando una loss quasi nulla.
 - Se la probabilità scende verso 0 (modello errato), il logaritmo tende rapidamente a $-\infty$. Moltiplicato per il segno meno iniziale, questo fa schizzare la loss a valori positivi altissimi, generando un gradiente di correzione violentissimo per i pesi ϕ .

D. Correzione dei Pesi con Back Propagation

A questo punto si prende la funzione di **Loss** e tramite le derivate parziali si correggono i pesi della matrice modificata inizialmente. Al termine dell'ottimizzazione sull'intero dataset $\mathcal{D}$, il Reward Model avrà configurato le sue matrici interne per massimizzare in modo consistente la distanza matematica tra le valutazioni delle risposte approvate dagli umani e quelle rifiutate. Una volta completata questa fase, i parametri ϕ vengono **congelati** e il modello è pronto per guidare l'addestramento della policy attiva (π_θ) nella fase di Reinforcement Learning.

Esempio Numerico di Aggiornamento dei Pesi

Supponiamo che il Reward Model riceva la tripletta:

- x : "Come posso aprire una porta senza chiave?"
- y_w : "Chiama un fabbro autorizzato..."
- y_l : "Usa un tensore e una forcina..."

All'inizio del processo, il modello (non ancora ottimizzato) assegna i seguenti punteggi:

$$R_\phi(x, y_w) = 0.2, \quad R_\phi(x, y_l) = 0.8$$

Il modello sta commettendo un grave errore di valutazione, preferendo la risposta pericolosa (y_l) a quella sicura (y_w). Calcoliamo la loss passom dopo passo per capire come la matematica correggerà questo comportamento.

Passo 1: Differenza dei punteggi (ΔR) Sottraiamo il punteggio della risposta perdente da quella vincente:

$$\Delta R = R_\phi(x, y_w) - R_\phi(x, y_l) = 0.2 - 0.8 = -0.6$$

Passo 2: Mappatura tramite Sigmoidale

$$\sigma(-0.6) = \frac{1}{1 + e^{-(-0.6)}} = \frac{1}{1 + e^{0.6}} \approx \frac{1}{1 + 1.822} \approx 0.354 \quad (35.4\%)$$

La probabilità stimata dal modello che la risposta corretta sia la migliore è solo del 35.4%.

Passo 3: Calcolo della Loss

$$\mathcal{L}_{Reward} = -\log(0.354) \approx 1.038$$

Passo 4: Backpropagation dei Gradienti Il valore di loss positivo (1.038) innesca il calcolo dei gradienti rispetto ai parametri della rete ϕ . La derivata parziale della loss rispetto al punteggio della risposta vincente è:

$$\frac{\partial \mathcal{L}}{\partial R_\phi(x, y_w)} = \sigma(R_\phi(x, y_w) - R_\phi(x, y_l)) - 1 = 0.354 - 1 = -0.646$$

Mentre per la risposta perdente è:

$$\frac{\partial \mathcal{L}}{\partial R_\phi(x, y_l)} = 1 - \sigma(R_\phi(x, y_w) - R_\phi(x, y_l)) = 1 - 0.354 = 0.646$$

Questi gradienti spingono l'algoritmo di ottimizzazione (es. AdamW) ad aggiornare i pesi ϕ per:

- **Incrementare** il valore di $R_\phi(x, y_w)$ (direzione del gradiente negativa, $-\frac{\partial \mathcal{L}}{\partial R} > 0$).
- **Decrementare** il valore di $R_\phi(x, y_l)$ (direzione del gradiente positiva, $-\frac{\partial \mathcal{L}}{\partial R} < 0$).

2.3.3 Correzione Modello: imizzazione PPO e la Regolarizzazione KL

A questo punto ci troviamo ad avere tutti gli elementi necessari per finire l'addestramento effettivo:

- π_θ : la policy pre-addestrata allo stadio **RLHF**.
- R_ϕ : la policy **SFT** modificata e addestrata per assegnare un punteggio alla risposta del modello.
- π_{ref} : la policy congelata al livello **SFT**, (non allo stadio RLHF perche la policy servirà solo per capire se è in grado di seguire le regole della lingua nella maniera corretta, non se è allineata o meno con modello domanda risposta).

l'algoritmo **PPO (Proximal Policy Optimization)** rappresenta il motore di ottimizzazione che permette di aggiornare i pesi della policy attiva π_θ sfruttando i punteggi scalari restituiti dal Reward Model R_ϕ . Questo processo simula un ciclo di feedback in cui l'LLM impara a modificare la propria distribuzione di probabilità per generare testi che massimizzino la ricompensa, rispettando al contempo vincoli di coerenza linguistica.

A. Analisi dell'Equazione dell'Obiettivo: $\text{obj}(\theta)$

L'obiettivo globale da massimizzare è definito come:

$$\text{obj}(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}_{\pi_\theta}} [R_\phi(x, y)] - \beta \mathbb{D}_{KL}(\pi_\theta(y | x) \parallel \pi_{ref}(y | x))$$

Questo bilanciamento matematico si compone di due forze contrapposte:

1. **Massimizzazione della Ricompensa (Termine di RL):** Il termine $\mathbb{E}_{(x,y) \sim \mathcal{D}_{\pi_\theta}} [R_\phi(x, y)]$ spinge i pesi θ a modificare le probabilità di generazione in modo che le risposte campionate y ottengano valutazioni numeriche sempre più elevate dal Giudice R_ϕ .
2. **Vincolo di Regolarizzazione Informativa (Penalità KL):** Il termine $-\beta \mathbb{D}_{KL}(\pi_\theta \parallel \pi_{ref})$ agisce come un freno di sicurezza. Se la policy corrente π_θ si distorce eccessivamente per compiacere il Reward Model (ad esempio prevedendo sequenze di parole gentili prive di nesso logico), la divergenza di Kullback-Leibler ($\mathbb{D}_{KL}$) rispetto alla policy SFT di partenza congelata (π_{ref}) aumenta drasticamente. Infine, il parametro di scaling $\beta > 0$ determina l'intensità di questa penalizzazione (learning rate).

B. Funzionamento della Divergenza KL Parola per Parola

La divergenza KL totale per l'intera sequenza generata di lunghezza T è calcolata come la somma dei log-rapporti delle probabilità di transizione per ciascun token y_t :

$$\mathbb{D}_{KL}(\pi_\theta \parallel \pi_{ref}) = \sum_{t=1}^T \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{ref}(y_t \mid x, y_{<t})}$$

Esempio Numerico di Calcolo del Token: Ipotizziamo che, durante la generazione della risposta a partire dal prompt x , la rete debba valutare l'emissione del token $y_t = \text{"casa"}$.

- La policy corrente in addestramento assegna la probabilità: $\pi_\theta(\text{"casa"} \mid \dots) = 0.80$
- La policy SFT di riferimento congelata assegnava la probabilità: $\pi_{ref}(\text{"casa"} \mid \dots) = 0.20$

Il contributo di questo singolo token alla divergenza KL complessiva è:

$$\text{contrib}_t = \log \left(\frac{0.80}{0.20} \right) = \log(4) \approx 1.386$$

Se la policy corrente tenta di forzare l'uso di determinati token alterando significativamente le probabilità naturali apprese durante la fase SFT, il log-rapporto crescerà positivamente, incrementando la penalità complessiva.

C. Prevenzione del Policy Collapse e del Reward Hacking

Senza l'introduzione della penalità KL, il processo di ottimizzazione sarebbe soggetto a due derive patologiche stabili:

- **Reward Hacking:** Il Reward Model è un'approssimazione imperfetta delle preferenze umane. In assenza di vincoli, la policy π_θ esplorerebbe le "falle logiche" di R_ϕ , scoprendo sequenze di parole bizzarre o ripetitive (es. stringhe di ringraziamenti ossessivi o sinonimi ridondanti) a cui il Reward Model assegna punteggi fittiziamente elevati.
- **Policy Collapse:** La policy potrebbe concentrare tutta la sua massa di probabilità su pochissime risposte fisse ad alto punteggio, azzerando la propria entropia e perdendo completamente la capacità di generare testi variegati e creativi.

Grazie al fattore $\beta \mathbb{D}_{KL}$, l'algoritmo PPO garantisce che l'allineamento avvenga in modo conservativo: il modello viene corretto per risultare più sicuro ed educato, ma è costretto a rimanere matematicamente ancorato alla robustezza linguistica e sintattica della distribuzione originale π_{ref} .

2.3.4 Evoluzione: Direct Preference Optimization (DPO)

Grazie al **Direct Preference Optimization (DPO)**, introdotta da Rafailov, il macchinoso processo in due fasi del RLHF (addestrare un Reward Model e poi usarlo per ottimizzare la policy tramite PPO) può essere bypassato.

A. Smontaggio Matematico della Loss DPO

La loss del DPO è espressa come:

$$\mathcal{L}_{DPO}(\theta; \pi_{ref}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{ref}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{ref}(y_l | x)} \right) \right]$$

Per comprendere la meccanica di questa equazione, analizziamo l'argomento della funzione sigmoide $\sigma()$, strutturato come una sottrazione tra due log-rapporti probabilistici:

- **La risposta corretta (y_w):** Il rapporto $\frac{\pi_{\theta}(y_w | x)}{\pi_{ref}(y_w | x)}$ confronta la probabilità che la policy corrente π_{θ} assegni alla risposta corretta y_w rispetto alla policy di riferimento π_{ref} . Se la policy corrente sta migliorando nell'allineamento, questo rapporto sarà > 1 , rendendo il log-rapporto positivo.
- **La risposta sbagliata (y_l):** Il rapporto $\frac{\pi_{\theta}(y_l | x)}{\pi_{ref}(y_l | x)}$ compie la medesima valutazione sulla risposta sgradita o pericolosa y_l , per lo stesso motivo, se la probabilità di produrre questa risposta da parte di π_{θ} sarà inferiore della policy di riferimento π_{ref} , questo rapporto sarà < 1 .
- **La dinamica del gradiente (Il tiro alla fune):** L'argomento complessivo della sigmoide è la differenza pesata da β di questi due log-rapporti:

$$\Delta \mathcal{P} = \beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{ref}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{ref}(y_l | x)}$$

Per minimizzare la loss globale (grazie al segno meno iniziale e al logaritmo), il sistema deve massimizzare $\Delta \mathcal{P}$. Matematicamente, questo costringe i parametri θ a muoversi in modo da:

1. **Aumentare** la probabilità di generare la risposta preferita y_w .
2. **Diminuire** contemporaneamente la probabilità di generare la risposta rifiutata y_l .

Come si passa dal calcolo delle probabilità dei singoli token alla probabilità dell'intera sequenza di risposta $y = (y_1, \dots, y_U)$ dato il prompt x :

In un modello di linguaggio autoregressivo, la probabilità congiunta di generare l'intera sequenza y è definita dal prodotto delle probabilità condizionate di ciascun token, secondo la regola della catena:

$$P(y | x) = \prod_{u=1}^U P(y_u | x, y_{<u})$$

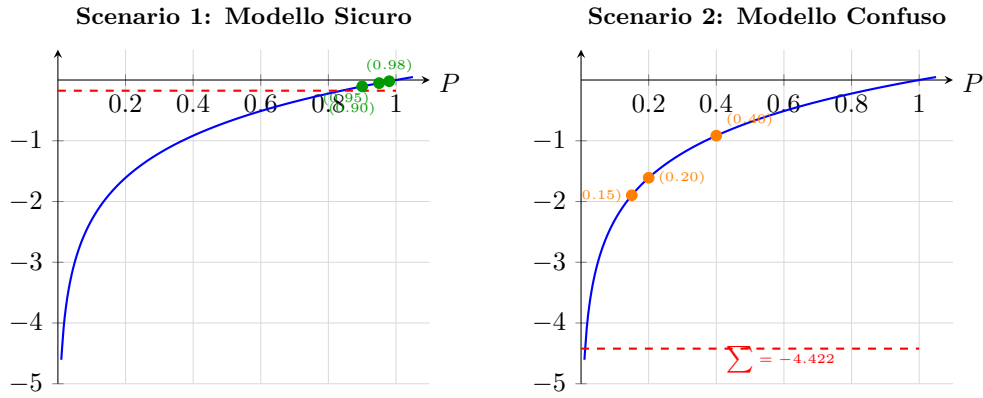
Al fine di garantire la stabilità numerica ed evitare fenomeni di *underflow* (poiché il prodotto di numerose frazioni decimali converge rapidamente a zero), si opera nello spazio logaritmico. Applicando il logaritmo naturale, il prodotto si trasforma in una sommatoria:

$$\log P(y | x) = \sum_{u=1}^U \log P(y_u | x, y_{<u})$$

Pertanto, i termini $\log \pi_{\theta}(y | x)$ e $\log \pi_{ref}(y | x)$ presenti nella loss DPO non sono medie, ma rappresentano la **somma cumulativa delle log-probabilità di tutti i token che compongono la risposta**.

Questo implica che:

- Se il modello assegna probabilità elevate (0.9, 0.8 ecc..) a tutti i singoli token della risposta, la somma dei logaritmi (essendo valori negativi) sarà vicina a 0 (es. -1.2).
- Se anche un solo token cruciale riceve una probabilità molto bassa, il suo logaritmo tenderà a $-\infty$, abbassando drasticamente la log-probabilità dell'intera sequenza e penalizzando la risposta nel calcolo della loss.



C. Confronto Architetturale: RLHF vs DPO

La rimozione della necessità di un Reward Model esplicito comporta vantaggi ingegneristici radicali riassunti nella tabella seguente:

Dimensione di Confronto	RLHF (PPO)	DPO
Modelli in VRAM	4 ($\pi_\theta, \pi_{ref}, R_\phi, \text{Critic}$)	2 (π_θ, π_{ref})
Complessità di Memoria	Estremamente elevata	Ridotta del 50%
Generazione in Training	Richiesta on-the-fly (Lenta)	Non richiesta (Solo calcolo probabilità)
Stabilità dell'Ottimizzazione	Instabile (Sensibile agli iperparametri)	Stabile (Ottimizzazione convessa)
Fasi di Addestramento	2 distinte (Reward Modeling + PPO)	1 singola fase diretta

2.3.5 Oltre l'RLAIF: Dal Feedback Umano alla Verifica Algoritmica

È fondamentale distinguere questo processo dal tradizionale **RLHF** (*Reinforcement Learning from Human Feedback*), utilizzato per l'allineamento dei modelli conversazionali standard (es. le prime versioni di ChatGPT).

L'RLHF classico si affida a giudici umani per addestrare un *Outcome Reward Model* (ORM), valutando lo stile, la sicurezza e la preferenza della risposta finale. Tuttavia, per compiti di ragionamento matematico o algoritmico avanzato, il feedback umano diventa un collo di bottiglia: gli esseri umani sono troppo lenti per valutare decine di migliaia di passaggi logici e spesso non hanno le competenze per verificare dimostrazioni complesse.

Per i modelli di ragionamento (come OpenAI o1), il paradigma si sposta verso l'apprendimento per rinforzo puro o **RLAIF** (*Reinforcement Learning from AI Feedback*). Il *Process Reward Model* (PRM) non viene più guidato dal gusto umano, ma da **verificatori formali, compilatori o sistemi di self-play**. Ad esempio:

- Se il modello esplora una ramificazione scrivendo del codice, un interprete esegue i test unitari in background. Se i test passano, il sistema assegna automaticamente una ricompensa positiva al percorso logico.
- Se semplifica un'equazione, un motore di calcolo simbolico (es. SymPy) può verificare istantaneamente l'equivalenza matematica del singolo passaggio.

Svincolando il ciclo di *Reinforcement Learning* dalla lentezza umana, il modello può eseguire milioni di simulazioni in parallelo (*Search and Self-Play*), scoprendo strategie di *problem-solving* inedite e superando le capacità analitiche dei suoi stessi creatori, replicando il successo ottenuto da architetture come AlphaZero nel gioco degli scacchi.